

Improved Differential Meet-In-The-Middle Cryptanalysis on SIMON and Piccolo (Full Version)

Weiqing Deng, Jianing Zhang, and Haoyang Wang^(✉)

Shanghai Jiao Tong University, Shanghai, China
{sdzwyq233,zhangjn,haoyang.wang}@sjtu.edu.cn

Abstract. Differential meet-in-the-middle (MITM) cryptanalysis, introduced by Boura *et al.* at CRYPTO 2023, has been demonstrated to be an effective technique for analyzing the security of block ciphers. In this paper, we introduce an improved parallel partitioning technique, and incorporate it into a new framework with a flexible key recovery strategy. This framework is applicable to both SPN and Feistel ciphers. We apply the new framework to SIMON and Piccolo-128 for demonstration. In particular, we also develop an MILP-based tool for searching full attacks on Piccolo-128. For SIMON48/96, we propose the best attack, reaching 26 rounds against 25 rounds previously. For other versions of SIMON, we extend the best previous differential attacks by one or two rounds. In the case of Piccolo-128, while no current differential attacks show promising results, our differential MITM attack reaches 15 rounds, matching the same number of rounds of the best impossible differential attack.

Keywords: differential meet-in-the-middle cryptanalysis · key recovery · parallel partitioning · SIMON · Piccolo-128

1 Introduction

Differential meet-in-the-middle (MITM) cryptanalysis, first introduced by Boura *et al.* at CRYPTO 2023 [8], combines the ideas of differential [7] and MITM attacks. This cryptanalysis method can be viewed as either an extension of the middle part of a MITM attack by adding more rounds, or as a novel approach to the key recovery in differential attacks. Benefiting from the MITM phase, techniques commonly used in MITM attacks can be applied in this new cryptanalysis. This novel cryptanalysis has yielded effective results on ciphers such as AES-256 [2] and SKINNY-128-384 [6].

Many improvements have been proposed for differential MITM attacks. In the original paper [8], Boura *et al.* introduced two crucial techniques: the parallel partitioning technique and the time-data-memory trade-off, both of which significantly enhanced the applicability of the basic differential MITM attack. In [3], Ahmadian *et al.* made a pioneering advance by incorporating truncated differentials, a state-test technique, and a probabilistic key-recovery method, while refining the parallel partitioning technique. These innovations led to the best-known attacks on CRAFT and SKINNY. Song *et al.* conducted an in-depth study

of differential MITM cryptanalysis in [20], introducing a flexible key guessing strategy and a generalized differential MITM attack (GDMA).

Among the existing techniques of differential MITM attacks, the parallel partitioning [8] is the most significant, as it allows gaining an additional round to the attack, offering a notable advantage over classical differential attacks. In [3], the authors improved this technique by exploring linear relations between the ciphertext and the internal state. However, their approach needs an in-depth study of specific SPN ciphers, which limits its broader applicability.

While most research on differential MITM attacks focuses on substitution-permutation network (SPN) ciphers, such as those discussed in [8,21,9,3,20,14], little attention has been given to such attacks against Feistel ciphers. To the best of our knowledge, only one work [11] simply applies the differential MITM attack to the Feistel ciphers *SLIM* and *LBCIoT*. Thus, it is essential to investigate whether the differential MITM attack exhibits distinct properties when applied to Feistel ciphers.

Our Contributions. In this paper, we propose an improved differential MITM attack. First, we improve the parallel partitioning technique in a more general way. This improvement enables a one-round extension for SPN and Feistel ciphers with a full key addition, and a two-round extension for those with a partial key addition. Second, building on the improved parallel partitioning technique, we incorporate the flexible key recovery strategy introduced in [20] into one framework to improve the differential MITM attack. Our framework has three features:

- An extended part in the end, enabling more rounds.
- A flexible key recovery that reduces data complexity, thereby preventing the need for a full-codebook attack.
- Applicable to both SPN and Feistel ciphers.

To demonstrate the effectiveness of our new framework, we apply it to *SIMON* and *Piccolo*. Specifically, for *Piccolo*, we develop a Mixed Integer Linear Programming (MILP)-based automatic tool to search full differential MITM attacks. The main attack results are summarized in Tables 1 and 2.

- For *SIMON*32/64, we achieve a 24-round differential MITM attack. Our attack matches the number of rounds in the previous best attack, but with a slightly lower time complexity.
- For *SIMON*48/96, we obtain a 26-round differential MITM attack, which outperforms the best previous attack by 1 round.
- For *SIMON*64/128, our differential MITM attack reaches 32 rounds, outperforming the best current differential attacks by 1 round.
- For *Piccolo*-128, we mount a 14-round attack with data complexity of 2^{54} , which demonstrates the effectiveness of our new framework, including the improved parallel partitioning technique and the flexible key guessing strategy.

- For **Piccolo-128**, we obtain a 15-round attack, matching the best known impossible differential attack. Note that no differential attacks exhibited good results previously.

Table 1. Summary of the best overall attacks and differential-like attacks against **SIMON** in the single-key setting. Memory complexity is measured in bytes. Time complexity is measured in encryptions.

Cipher	#Rounds	Method	Data	Time	Memory	Reference
SIMON32/64	23	Linear	$2^{30.59}$	2^{50}	—	[1]
	24	Linear	$2^{30.59}$	$2^{63.9}$	—	[1]
	22	Differential	$2^{32\dagger}$	$2^{58.76}$	—	[18]
	23	Diff-MITM	$2^{32\dagger}$	$2^{63.52}$	$2^{51.32}$	Sect. 4.1
	24	Diff-MITM	$2^{32\dagger}$	$2^{63.8}$	$2^{66.57}$	Sect. 4.2
SIMON48/96	25	Linear	$2^{47.92}$	$2^{90.96}$	2^{86}	[24]
	25	Linear	$2^{47.92}$	$2^{88.28}$	—	[10]
	24	Differential	$2^{48\dagger}$	$2^{78.99}$	—	[18]
	26	Diff-MITM	2^{47}	$2^{95.31}$	$2^{96.14}$	Sect. 4.3
SIMON64/128	32	Linear	$2^{63.84}$	2^{120}	2^{106}	[24]
	31	Differential	$2^{63.3}$	2^{120}	—	[23]
	32	Diff-MITM	$2^{63.8}$	2^{127}	$2^{131.4}$	App. D.2

[†] Requires the full codebook.

Our results demonstrate that differential MITM attacks offer advantages over classical differential attacks in certain scenarios. Furthermore, allowing for flexible key guessing strategies significantly enhances the effectiveness of the differential MITM attack.

Organization. In Section 2, we provide a brief overview of the basic differential MITM attacks, along with some improvements. In Section 3, we present an improved framework for differential MITM attacks. To demonstrate the improved framework, we present the improved attacks on **SIMON** and **Piccolo-128** in Sections 4 and 5, respectively. Finally, the paper is concluded in Section 6.

2 Differential Meet-In-The-Middle Cryptanalysis

In this section, we provide an overview of the basic differential MITM attack and some improvements provided in [8].

Table 2. Summary of the best attacks against **Piccolo-128** in the single-key setting together with the results presented in this paper. Memory complexity is measured in **bytes**. Time complexity is measured in **encryptions**.

Method	#Rounds	Pre/Post [*]	Data	Time	Memory	Ref.
MITM	16	Post	2^{48}	2^{123}	$2^{113.49}$	[22]
	17	Post	2^{48}	$2^{126.87}$	$2^{125.99}$	[22]
	17	Pre	2^{44}	2^{126}	$2^{125.49}$	[16]
	18	Post	2^{52}	$2^{126.63}$	$2^{125.29}$	[15]
	21	None	$2^{64\dagger}$	2^{121}	2^6	[12]
ID [‡]	15	Post	$2^{58.7}$	$2^{125.4}$	2^{64}	[4]
Diff-MITM	14	None	2^{54}	2^{124}	$2^{79.17}$	Sect. 5.3
	15	None	$2^{64\dagger}$	$2^{121.56}$	2^{68}	Sect. 5.2
	15	None	2^{60}	$2^{125.56}$	2^{64}	Sect. 5.2

^{*} Pre: Pre-whitening Key; Post: Post-whitening Key; None: No whitening key.

[†] Requires the full codebook.

[‡] ID: Impossible differential.

2.1 Basic Differential MITM Attack

Suppose that an n -bit block cipher E is composed of three sub-ciphers $E = E_{out} \circ E_p \circ E_{in}$ with r_{in} , r_p , r_{out} rounds, respectively. Let Δ_{in} and Δ_{out} be the input difference and the output difference of a differential distinguisher with a probability of 2^{-p} over E_p . The framework of the basic differential MITM attack is shown in Figure 1.

For the procedure of the attack, we first randomly choose a plaintext-ciphertext pair (P, C) . The goal is to find another pair $(\tilde{P}, \tilde{C})$ that satisfies the following conditions:

- $E_{in}(P) \oplus E_{in}(\tilde{P}) = \Delta_{in}$
- $E_{out}^{-1}(C) \oplus E_{out}^{-1}(\tilde{C}) = \Delta_{out}$

We denote k_{in} (resp. k_{out}) as the minimum set of key bits that need to be guessed to compute $\tilde{P}$ (resp. $\tilde{C}$) from P (resp. C).

- *The Upper Part.* For each possible guess i of k_{in} , compute $\tilde{P}$ from P such that it satisfies $E_{in}(P) \oplus E_{in}(\tilde{P}) = \Delta_{in}$. Ask for its ciphertext $\tilde{C}$, and store it in a hash table H , i.e., $H[\tilde{C}] = H[\tilde{C}] \cup i$. There will be $2^{|k_{in}|}$ elements in the hash table.
- *The Lower Part.* Similarly, for each possible guess j of k_{out} , compute $\tilde{C}$ from C such that it satisfies $E_{out}^{-1}(C) \oplus E_{out}^{-1}(\tilde{C}) = \Delta_{out}$. We will obtain $2^{|k_{out}|}$ candidates for $\tilde{C}$ and j .

For the correct key guess, the differential $\Delta_{in} \rightarrow \Delta_{out}$ will happen with probability 2^{-p} . After selecting 2^p different plaintext-ciphertext pairs and performing the above steps, we expect to find a pair that satisfies the distinguisher. We check the collisions between the ciphertexts computed from the upper part

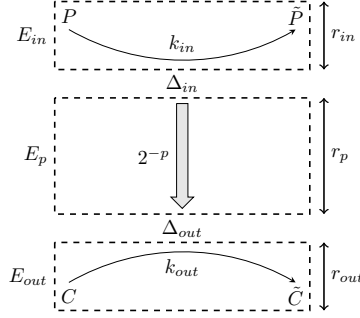


Fig. 1. Framework of the basic differential meet-in-the-middle attack.

and the lower part. Each collision suggests a key candidate (i, j) . For each (P, C) , the number of expected collisions is $2^{|k_{in} \cup k_{out}| - n}$.

Complexity Analysis. The time complexity can be estimated as

$$\mathcal{T} = 2^p \times (2^{|k_{in}|} + 2^{|k_{out}|}) + 2^{|k_{in} \cup k_{out}| - n + p} + 2^{k - n + p},$$

where the first term represents the computations performed in the upper and lower parts, the second corresponds to the number of expected key candidates, and the last one is the computation for exhaustive search.

For the memory complexity, the dominant factor is the hash table. By guessing $k_{in} \cap k_{out}$ in advance, the memory complexity is thus

$$\mathcal{M} = 2^{\min(|k_{in}|, |k_{out}|) - |k_{in} \cap k_{out}|}.$$

The (naive) data complexity of this attack can be estimated as

$$\mathcal{D} = \min(2^n, 2^{p + \min(|k_{in}|, |k_{out}|)}).$$

2.2 Parallel Partitions for Layers with Partial Subkeys

Boura *et al.* [8] show that if the round key addition affects only m bits ($m < p$) in the internal state (see Fig. 2), i.e., there are $n - m$ pre-matched fixed bits between S_{r-1} and B , the differential MITM attack can be extended by one round. The main idea of the improvement is to only keep the ciphertexts that satisfy the following condition: we fix the $n - m$ bits that are not affected by the key addition in A and B , and collect all the 2^m possible values for A and B to construct the hash table and find collisions.

For all the 2^m possible values of A , we proceed with the lower part of the differential MITM attack, and obtain $2^{m + |k_{out}|}$ possible tuples $(S_{r-1}, \tilde{S}_{r-1}, j)$. In parallel, we perform the same process in the lower part for the 2^m possible values of B , and $2^{m + |k_{in}|}$ possible tuples $(B, \tilde{B}, i)$ are obtained. Hence, we expect to find

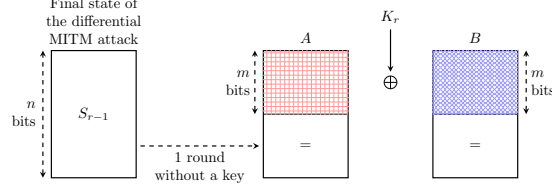


Fig. 2. Parallel partitioning technique introduced in [8].

$2^{2m+|k_{in}|+|k_{out}|}$ collisions, which is a factor of 2^m higher than before. However, the number of collisions does not actually increase, as explained below. Let us consider the filters when matching two tuples: $B = A \oplus K_r$ represents a m -bit filter since we have fixed $n - m$ bits between A and B , $\tilde{B} = \tilde{A} \oplus K_r$ represents a n -bit filter since there are no pre-matched fixed bits between $\tilde{A}$ and $\tilde{B}$. Besides, there is another $|k_{in} \cap k_{out}|$ -bit filter to match k_{in} and k_{out} . After these filters, the number of candidates becomes $2^{|k_{in} \cup k_{out}|+m-n}$.

Now, we repeat the procedure 2^{p-m} times since we expect to obtain one pair satisfying the distinguisher. There will be $2^{|k_{in} \cup k_{out}|+m-n} \times 2^{p-m} = 2^{|k_{in} \cup k_{out}|-n+p}$ candidates in total, same as the attack without parallel partitioning technique. Thus, we can apply this improvement without increasing the time complexity, if $p > m$.

2.3 Reducing Data with Imposed Conditions

Boura et al. [8] provide another method to reduce the data complexity for the basic differential MITM attack, especially when the attack requires the entire codebook. The idea is to fix x bits of both the plaintext P and the associated plaintext $\tilde{P}$. As a result, the data and memory complexities decrease by 2^{n-x} , at the cost of increasing time complexity by a factor of 2^x as the attack procedure has to be repeated 2^x times to get a correct pair. Considering the attributes of the differential MITM attack, we can derive the following two inequalities for x : $p + x \leq n - x$ and $2^{p+x}(2^{|k_{in}|} + 2^{|k_{out}|})$.

When extra rounds are added at the end, i.e., utilizing the parallel partitioning technique, does the data-time-memory trade-off still hold? Consider a typical scenario where there are $n - m$ pre-matched fixed bits between S_{r-1} and B . If we fix these $n - m$ bits as constants, we need to repeat the attack procedure 2^{p-m} times. For a given S_{r-1} , we can still obtain the corresponding $\tilde{S}_{r-1}$ after guessing k_{out} . Therefore, the number of $(S_{r-1}, \tilde{S}_{r-1}, j)$ remains unchanged. However, when only a portion of the codebook is known, we expect to discard 2^{-x} of $(B, \tilde{B}, i)$. Consequently, we need to repeat the attack procedure 2^{x+p-m} times to recover the correct master key. In conclusion, the data-time-memory trade-off still holds when the parallel partitioning technique is used.

3 Improved Differential MITM Attack

In this section, we propose an improved version of the differential MITM attack by generalizing the parallel partitioning technique under the generalized differential MITM proposed in [20]. We show that it is possible to add an extra round in the case of full key addition, and add two rounds for attacking Feistel ciphers with a linear key schedule.

3.1 Framework of the Improved Differential MITM Attack

In [20], the authors introduced a generalized differential MITM attack that incorporates a flexible key guessing strategy, which has the advantage of reducing the data complexity. Based on this, we propose an improved framework that enhances the parallel partitioning technique. For ciphers that the round key addition affects the whole internal state, our new framework enables us to achieve an extra round; for Feistel ciphers with a linear key schedule, it is possible for us to add two rounds with our improved attack.

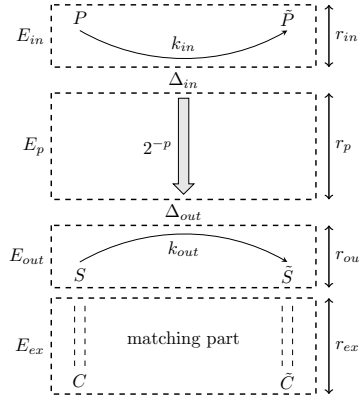


Fig. 3. Framework of the improved differential meet-in-the-middle attack. This framework is under the chosen-ciphertext scenario.

The framework is depicted in Fig. 3. For simplicity, we only consider the case of adding additional rounds in the end, the same applies if they are added at the beginning. We decompose an n -bit block cipher E into four components: $E_{in} \circ E_p \circ E_{out} \circ E_{ex}$. Let the number of rounds of E , E_{in} , E_p , E_{out} , and E_{ex} be denoted as r , r_{in} , r_p , r_{out} , and r_{ex} , respectively. Similarly, let the involved key bits of E_{in} , E_{out} , and E_{ex} be represented as k_{in} , k_{out} , and k_{ex} , respectively. Finally, suppose that the differential $\Delta_{in} \rightarrow \Delta_{out}$ over E_p has a probability of 2^{-p} .

We first assume that $|\Delta_P| = |\Delta_S| = n$, which is the general case since we aim to attack as many rounds as possible¹. Given that $|\Delta_P| = n > p$, we can utilize a partial structure to reduce the data complexity, as is common in classical differential attacks. Specifically, we build a partial structure ST_1 consisting of ciphertexts and the corresponding plaintexts by fixing $x = \frac{n-p-1}{2}$ bits in the ciphertext (that might have been active otherwise), thus the size of the structure is $2^{n-x} = 2^{(n+p+1)/2}$. This will lead to $\binom{2^{n-x}}{2} = 2^{n+p}$ pairs of plaintexts, of which about 2^p pairs will satisfy Δ_{in} . Therefore, we expect to obtain one pair of plaintexts that satisfies Δ_{out} .

Next, we focus on E_{ex} and construct another structure. Suppose there are $n - m$ (could be zero) pre-matched fixed bits between ciphertext C and the internal state S without the knowledge of k_{ex} . For example, when $r_{ex} = 1$, we have $n - m = 0$ for ciphers with full key addition, such as AES; $n - m = \frac{n}{2}$ for ciphers with partial key addition, such as Feistel ciphers and SKINNY. To ensure we do not miss any possible collisions, the following condition must hold: $E_{ex}^{-1}(ST_1) \subseteq ST_2$ for all k_{ex} . Thus, we can set $x' = \min(n - m, x)$ and collect $2^{n-x'}$ internal states S to construct the structure ST_2 , see Fig. 4.

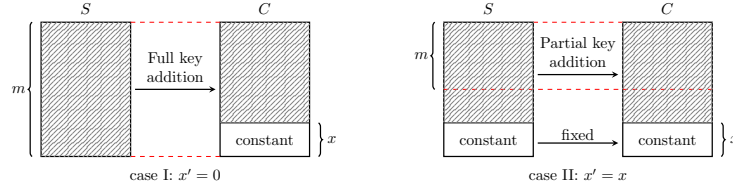


Fig. 4. Description of the second structure ST_2 .

The following notations are followed throughout the rest of the paper.

- k'_{in} (resp. k'_{out}) represents the part of k_{in} (resp. k_{out}) guessed in advance;
- k_{in}^* (resp. k_{out}^*) represents $k_{in} \setminus k'_{in}$ (resp. $k_{out} \setminus k'_{out}$);
- $k'_{\cap}$ (resp. $k'_{\cup}$) represents $k'_{in} \cap k'_{out}$ (resp. $k'_{in} \cup k'_{out}$);
- c'_{in}, c'_{out} and c'_{ex} represent the number of conditions that can be verified under k'_{in}, k'_{out} and $k'_{\cup}$, respectively;
- $c_{in}^* = |\Delta_P| - c'_{in} = n - c'_{in}$;
- $c_{out}^* = |\Delta_S| - c'_{out} = n - c'_{out}$;
- $c_{ex}^* = 2(n - x') - c'_{ex}$, since the total number of filtering bits in E_{ex} is $2(n - x')$.

Attack Procedure. The attack procedure is as follows and can also be found in Algorithm 1:

1. Prepare two structures, ST_1 and ST_2 , using the method described above.
2. Guess all possible values of $k'_{\cap}$.

¹ Our framework is still applicable without this assumption, but the exact complexity of the attack has to be checked case-by-case.

- (a) *The Upper Part.* Guess all possible values of i for $k'_{in} \setminus k'_\cap$, and partially encrypt each plaintext P in ST_1 using the guessed key bits. Generate $\binom{2^{n-x}}{2} \times 2^{-c'_{in}}$ pairs $(P, \tilde{P})$ after applying a c'_{in} -bit filter. For each pair, compute the corresponding ciphertexts $C = E(P)$ and $\tilde{C} = E(\tilde{P})$. Finally, store all tuples $(C, \tilde{C}, i)$ in a hash table H .
- (b) *The Lower Part.* Guess all possible values of j for $k'_{out} \setminus k'_\cap$, and partially decrypt each state S in ST_2 using the guessed key bits. Generate $\binom{2^{n-x'}}{2} \times 2^{-c'_{out}}$ pairs $(S, \tilde{S})$ after applying a c'_{out} -bit filter. For each tuple $(S, \tilde{S}, j)$, check the hash table H for collisions.
3. We expect to obtain $2^{|k'_{in} \cup k'_{out}| - c'_{in} - c'_{out} - c'_{ex}} \times \binom{2^{n-x}}{2} \times \binom{2^{n-x'}}{2} \times 2$ collisions² in total. For each collision, we extract $2^{|k_{in}^* \cup k_{out}^*| - c_{in}^* - c_{out}^* - c_{ex}^*}$ key candidates. We then exhaust the remaining key bits, and either test all candidates directly or update the key counters.

Complexity. The data complexity of this improved attack equals the size of ST_1 , i.e., $\mathcal{D} = |ST_1| = 2^{\frac{n+p+1}{2}}$. By leveraging the flexible key guessing strategy, we avoid the need for the entire codebook even when $|\Delta_P| = |\Delta_S| = n$.

The time complexity $\mathcal{T}$ consists of several parts:

- $\mathcal{T}_0 = \mathcal{D}$ for step 1;
- $\mathcal{T}_1 = \frac{r_{in}}{r} \times 2^{|k'_{in}| + n - x} + \binom{2^{n-x}}{2} \times 2^{|k'_{in}| - c'_{in}}$ for step 2 (a);
- $\mathcal{T}_2 = \frac{r_{out}}{r} \times 2^{|k'_{out}| + n - x'} + \binom{2^{n-x'}}{2} \times 2^{|k'_{out}| - c'_{out}}$ for step 2 (b);
- $\mathcal{T}_3 = 2^{|k'_{in} \cup k'_{out}| - c'_{in} - c'_{out} - c'_{ex}} \times \binom{2^{n-x}}{2} \times \binom{2^{n-x'}}{2} \times 2$ for the number of collisions in step 3;
- $\mathcal{T}_4 = \mathcal{T}_3 \times 2^{|k_{in}^* \cup k_{out}^*| - c_{in}^* - c_{out}^* - c_{ex}^*} = 2^{k-2n-(c'_{ex}-c_{ex}^*-1)} \times \binom{2^{n-x}}{2} \times \binom{2^{n-x'}}{2}$ for recovering the master key in step 3. Since $2^{c'_{ex}+c_{ex}^*-1} = 2^{2(n-x')-1} = \binom{2^{n-x'}}{2}$ and $\binom{2^{n-x}}{2} = 2^{n+p}$, we can deduce that $\mathcal{T}_4 = 2^{k-n+p}$.

The memory complexity includes the storage of the data, the hash table, and the key counter table if necessary. If we test all key candidates directly, the memory complexity can be estimated as:

$$\mathcal{M} = \max\{\mathcal{D}, \min\{\binom{2^{n-x}}{2} \times 2^{|k'_{in} \setminus k'_\cap| - c'_{in}}, \binom{2^{n-x'}}{2} \times 2^{|k'_{out} \setminus k'_\cap| - c'_{out}}\}\}.$$

Full Key Addition. When applying the improved attack to ciphers with full key addition (case I in Fig. 4), both $\mathcal{T}_2$ and $\mathcal{T}_3$ may increase a priori. We have the inequality $2^k > \mathcal{T}_2 > \binom{2^n}{2} \times 2^{|k'_{out}| - c'_{out}}$, which implies that $|k'_{out}| - c'_{out} < k - 2n + 1$. Therefore, it is possible to add one additional round to the attack in the case of full key addition, when the key size is larger than the state size, e.g. $k = 3n$.

² The number of expected collisions is multiplied by 2 because there are two possible cases: $C = E_{ex}(S)$ and $C = E_{ex}(\tilde{S})$.

3.2 Application to Typical Feistel Ciphers

For a Feistel block cipher with a linear key schedule, we can apply our improved framework to attack two more rounds, compared to the basic differential MITM attack. We introduce a method that allows us to set $r_{ex} = 2$ by generating a hash table that contains the full information of 2^n messages on one side by setting $x' = 0$.

In Fig. 7, we provide a typical two-round Feistel structure of E_{ex} , and our goal is to match $L_0||R_0$ with $L_2||R_2$. We would have the following equations:

- $L_1 = R_0 \oplus F(L_0) \oplus k_0, R_1 = L_0,$
- $L_2 = R_1 \oplus F(L_1) \oplus k_1, R_2 = L_1.$

Substituting $R_0 \oplus F(L_0) \oplus k_0$ for L_1 prevents us from splitting k_0 across both sides of the equation, as F is a nonlinear function. Therefore, we replace L_1 with R_2 , resulting in the following equations:

- $R_2 = R_0 \oplus F(L_0) \oplus k_0,$
- $L_2 = L_0 \oplus F(R_2) \oplus k_1,$

where k_0 and k_1 can be derived linearly from k_{in} , k_{out} , and $k \setminus (k_{in} \cup k_{out})$.

However, these equations cannot directly match $L_0||R_0$ with $L_2||R_2$ in the original parallel partitioning method, as there are no pre-matched fixed bits. Consequently, partitioning the entire codebook is not feasible in this case. As discussed in Sect. 3.1, the upper and lower parts of the attack do not need to share the same structure, meaning that it is not necessary to have $E_{ex}^{-1}(ST_1) = ST_2$. Thus, we can set $x' = 0$ and construct a hash table by enumerating all 2^n internal states S in the lower part (when $|k_{out}|$ is smaller). We then enumerate 2^p different messages in the upper part to find matches. According to the time complexity analysis, we observe that the value of x' does not affect $\mathcal{T}_4$, which is usually the bottleneck. In this case, the data complexity of the improved differential MITM attack is $\mathcal{D} = 2^{\frac{n+p+1}{2}}$. The memory complexity increased to $2^{n+|k_{out}|}$. This approach trades increased memory consumption for additional rounds in Feistel ciphers. The time complexity $\mathcal{T}$ is given by³:

- $\mathcal{T}_0 = \mathcal{D} = 2^{\frac{n+p+1}{2}};$
- $\mathcal{T}_1 = \mathcal{D} \times \frac{r_{in}}{r} \times 2^{|k_{in}|} + 2^{p+|k_{in}|};$
- $\mathcal{T}_2 = 2^{n+|k_{out}|} \left(\frac{r_{out}}{r} + \frac{1}{2} \right);$
- $\mathcal{T}_3 = 2^{|k_{in} \cup k_{out}| - n + p};$
- $\mathcal{T}_4 = 2^{k-n+p}.$

The core idea presented in this subsection can also be applied to the basic differential MITM attack. Here we demonstrate the associated complexity: the data complexity $\mathcal{D} = 2^n$, the memory complexity $\mathcal{M} = 2^{n+|k_{out}|}$, and the time complexity $\mathcal{T} = \frac{2r_{in}}{r} 2^{p+|k_{in}|} + \frac{2r_{out}}{r} 2^{n+|k_{out}|} + 2^{k-n+p}$.

We demonstrate this method with attacks against SIMON in Sect. 4.2, Sect. 4.3, and App. D.2.

³ Let $k'_{in} = k_{in}$ and $k'_{out} = k_{out}$.

4 Application on SIMON

We propose several differential MITM attacks against SIMON within our improved framework. The specification of SIMON is provided in Appendix C.

4.1 Differential MITM Attack against 23-round SIMON32/64

In this attack, we use a 14-round differential trail $(0x0, 0x1) \rightarrow (0x400, 0x100)$ for SIMON32 from [17], with a probability of $p = 2^{-30.76}$. By adding 4 rounds before and 4 rounds after the distinguisher, we obtain a 22-round attack. This can be further extended by one additional round using the parallel partitioning method without extra computations.

Key Guessing Part. By applying the algorithm for automatic detection of involved key bits introduced in [8], we can determine k_{in} and k_{out} for our attack⁴, as listed in Table 6.

Since $r_{in} = r_{out} = 4$, we can deduce that $|k_{in}| = |k_{out}| = 27$ based on the key schedule of SIMON. Given that the key schedule of SIMON is linear, each bit in k_{in} and k_{out} can be expressed as a linear combination of a subset of master key bits. We then arrange these 54 vectors into a matrix and perform Gaussian elimination. The matrix has a rank of 54. Therefore, $|k_{in} \cap k_{out}| = 0$, and we must guess 10 additional bits, $k_{re} = k[16, 18, 19, 20, 28, 29, 32, 33, 34, 37]$, to recover the full master key.

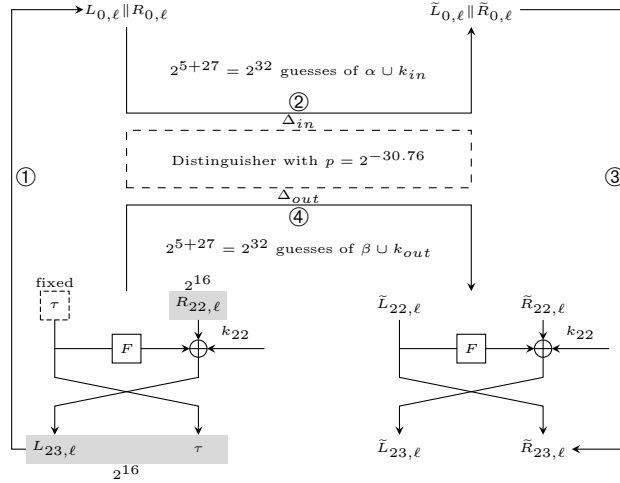


Fig. 5. 23-Round Differential MITM Attack of SIMON.

⁴ There is no need to guess k_3 and k_{18} due to the Feistel structure.

Attack Procedure. We now describe our differential MITM attack against 23-round SIMON32/64.

1. Ask for the encryption of the whole codebook. That is, for any plaintext $P = L_0 \| R_0$, we query for its corresponding ciphertext $C = L_{23} \| R_{23}$.
 2. Pick a 16-bit constant τ . Enumerate all the 2^{16} ciphertexts $L_{23,\ell} \| \tau$ and collect 2^{16} plaintext-ciphertext pairs $(L_{0,\ell} \| R_{0,\ell}, L_{23,\ell} \| \tau)$ (see ① in Fig. 5).
 3. Guess any 5 bits α of k_{re} and execute the upper part. For each plaintext-ciphertext pair, compute $\tilde{L}_{0,\ell}^i \| \tilde{R}_{0,\ell}^i$ for all possible guess i of k_{in} such that the state difference after the 4th round function is Δ_{in} . Look up the codebook to obtain $\tilde{L}_{23,\ell}^i \| \tilde{R}_{23,\ell}^i = E(\tilde{L}_{0,\ell}^i \| \tilde{R}_{0,\ell}^i)$. Thus, we will get $2^{16+5+27} = 2^{48}$ tuples in the form of $(L_{23,\ell} \| \tau, \tilde{L}_{23,\ell}^i \| \tilde{R}_{23,\ell}^i, \alpha, i)$. Store them in a hash table H (see ② and ③ in Fig. 5).
 4. Guess the remaining 5 bits β of k_{re} ($k_{re} = \alpha \cup \beta$) and execute the lower part. Enumerate all the 2^{16} internal states $\tau \| R_{22,\ell}$. For each possible guess j of k_{out} , compute $2^{16+5+27} = 2^{48}$ tuples in the form of $(\tau \| R_{22,\ell}, \tilde{L}_{22,\ell}^j \| \tilde{R}_{22,\ell}^j, \beta, j)$ such that the difference on the state before the 19th round function is Δ_{out} (see ④ in Fig. 5).
 5. Execute the matching part. The match is performed on several equations for each tuple in the lower part:
 - We derive the first equation by describing the last round: the internal state $\tau \| R_{22,\ell}$ transforms into $L_{23,\ell} \| \tau$ after the 23rd round function, indicating that $L_{23,\ell} = R_{22,\ell} \oplus F(\tau) \oplus k_{22}$. Since the key schedule of SIMON is linear, k_{22} can be expressed as $f(k_{in}) \oplus g(k_{out}) \oplus h(k_{re})$, where f , g , and h are linear functions. Thus, the equation can be rewritten as $L_{23,\ell} \oplus f(i) \oplus F(\tau) \oplus h(\alpha) = R_{22,\ell} \oplus g(j) \oplus h(\beta)$, representing a 16-bit filter.
 - The transformation from $\tilde{L}_{22,\ell}^j \| \tilde{R}_{22,\ell}^j$ to $\tilde{L}_{23,\ell}^i \| \tilde{R}_{23,\ell}^i$ indicates that we have found the right pair that satisfies $\Delta_{in} \rightarrow \Delta_{out}$. We can express this with two equations: $\tilde{R}_{23,\ell}^i = \tilde{L}_{22,\ell}^j$ and $\tilde{L}_{23,\ell}^i \oplus f(i) \oplus F(\tilde{R}_{23,\ell}^i) \oplus h(\alpha) = \tilde{R}_{22,\ell}^j \oplus g(j) \oplus h(\beta)$. The two equations represent a 32-bit filter.
- Thus, we expect to obtain $2^{48+48-(16+32)} = 2^{48}$ matches, and the key candidates can be derived from i, j, α , and β . Additional tests, such as encrypting a random message for each current key candidate, are performed to find the correct one.
6. Repeat from Step 2 until the master key is retrieved.

Complexity. The attack must be repeated enough times to ensure that at least one pair satisfies the differential condition. In Step 2, we generate 2^{16} pairs, and the probability of the differential is $2^{-30.76}$. Consequently, the procedure needs to be repeated approximately $2^{14.76}$ times.

The data complexity is $\mathcal{D} = 2^{32}$, as it requires the entire codebook. The memory complexity is approximately $\mathcal{M} = 2^{48}$ 80-bit words to store the hash table. Finally, the time complexity is $\mathcal{T} = 2^{32} + 2^{14.76} \cdot (2^{48} \times \frac{8}{23} + 2^{48} \times \frac{8}{23} + 2^{48}) = 2^{63.52}$ encryptions.

General Formula of Differential MITM Attack against SIMON n /2 n . When applying differential MITM cryptanalysis combined with parallel partitioning on a SIMON cipher with a key size of $k = 2n$, we can derive a general formula under specific conditions. Specifically, if we can find a differential trail such that the following conditions are met: $|k_{in} \cap k_{out}| = 0$, $|k_{in}| \leq n$, and $|k_{out}| \leq n$, we can always split the remaining key bits k_{re} into two parts, α and β , such that $|k_{in}| + |\alpha| = |k_{out}| + |\beta| = n$.

This results in an attack with a time complexity lower than that of an exhaustive search. In such a case, the complexities can be expressed in general form as: $\mathcal{D} = 2^n$, $\mathcal{T} \approx 2^{p+n}$ encryptions, and $\mathcal{M} = 2^{\frac{3n}{2}} \times \frac{5n}{2}$ bits.

4.2 Differential MITM Attack against 24-round SIMON32/64

Our differential MITM attack against 23-round SIMON32/64 can be extended to a 24-round attack by using the technique introduced in Sect. 3.2.

Attack Procedure.

1. Ask for the encryption of the whole codebook. That is, for any plaintext $P = L_{0,\ell} \| R_{0,\ell}$, we query for its corresponding ciphertext $C = L_{24,\ell} \| R_{24,\ell}$.
2. As in the attack in Sect. 4.1, we can express k_{22} as $f_0(k_{in}) \oplus g_0(k_{out}) \oplus h_0(k_{re})$ and k_{23} as $f_1(k_{in}) \oplus g_1(k_{out}) \oplus h_1(k_{re})$, where $f_{0,1}, g_{0,1}, h_{0,1}$ are all linear functions.
3. For each plaintext-ciphertext pair, guess arbitrary 4 bits α of k_{re} and all possible values i of k_{in} . Then, perform partial encryption to obtain $2^{n+|\alpha|+|k_{in}|} = 2^{32+4+27} = 2^{63}$ tuples in the form of $(L_{24,\ell} \| R_{24,\ell}, \tilde{L}_{24,\ell}^i \| \tilde{R}_{24,\ell}^i, \alpha, i)$ satisfying Δ_{in} . Compute the four elements below for each tuple then store them in a hash table.
 - $R_{24,\ell} \oplus f_0(i) \oplus h_0(\alpha)$
 - $L_{24,\ell} \oplus F(R_{24,\ell}) \oplus f_1(i) \oplus h_1(\alpha)$
 - $\tilde{R}_{24,\ell}^i \oplus f_0(i) \oplus h_0(\alpha)$
 - $\tilde{L}_{24,\ell}^i \oplus F(\tilde{R}_{24,\ell}^i) \oplus f_1(i) \oplus h_1(\alpha)$
4. Randomly choose $2^p = 2^{30.76}$ different values of $L_{22,\ell} \| R_{22,\ell}$. Guess the remaining 6 bits β of k_{re} and all possible values j of k_{out} . Then perform partial decryption to obtain $(L_{22,\ell} \| R_{22,\ell}, \tilde{L}_{22,\ell}^j \| \tilde{R}_{22,\ell}^j, \beta, j)$. Compute the four elements below and match them in the hash table ($2^{p+|\beta|+|k_{out}|} = 2^{30.76+6+27} = 2^{63.76}$ times in total):
 - $R_{22,\ell} \oplus F(L_{22,\ell}) \oplus g_0(j) \oplus h_0(\beta)$,
 - $L_{22,\ell} \oplus g_1(j) \oplus h_1(\beta)$,
 - $\tilde{R}_{22,\ell}^j \oplus F(\tilde{L}_{22,\ell}^j) \oplus g_0(j) \oplus h_0(\beta)$,
 - $\tilde{L}_{22,\ell}^j \oplus g_1(j) \oplus h_1(\beta)$.

The matching part indicates a 64-bit filter, thus we will obtain

$$2^{p+|k_{out}|+|\beta|+|k_{in}|+|\alpha|+n-2n} = 2^{p+k-n} = 2^{62.76}$$

key candidates. Perform additional encryption for each candidate to determine the correct master key.

Complexity. This attack achieves 24 rounds with a time complexity that remains approximately the same. The requirement for the entire codebook implies that the data complexity is $\mathcal{D} = 2^{32}$. The memory complexity is $\mathcal{M} = 2^{66.57}$, requiring 2^{63} 95-bit words to store the hash table. The time complexity is $\mathcal{T} = 2^{63} \times \frac{8}{24} + 2^{63.76} \times \frac{8}{24} + 2^{62.76} = 2^{63.8}$ encryptions.

4.3 Differential MITM Attack against 26-round SIMON48/96

We employ the CryptoSMT tool proposed in [13] to find a 16-round differential $(0x11, 0x40) \rightarrow (0x111, 0x40)$ for SIMON48 and calculate its probability. We found approximately 100,000 trails with probability ranging from 2^{-50} to 2^{-68} , and the resulting differential has a probability of $2^{-p} = 2^{-44.33}$.

Key Guessing Part. We add 4 rounds before and 4 rounds after the distinguisher. k_{in} and k_{out} for this attack are listed in Table 7.

We use the same method as described in Sect. 4.1 to verify that $|k_{in}| = 37$, $|k_{out}| = 51$, $|k_{re}| = 8$, and that $k_{in} \cap k_{out} = \emptyset$. Thus, we can set $|\alpha| = 8$ and $|\beta| = 0$. By applying a similar attack as described in Sect. 4.2, we can append two additional rounds, resulting in a 26-round attack.

Complexity. The data complexity is $\mathcal{D} = 2^{48}$, the time complexity is $\mathcal{T} = 2^{51+44.33} \times \frac{8}{26} + 2^{37+8+48} \times \frac{8}{26} + 2^{96-48+44.33} = 2^{94.31}$ encryptions, and the memory complexity $\mathcal{M} = 2^{93}$ 141-bit words.

Data-time-memory trade-off. As explained in Sect. 2.3, we can force one bit in the plaintext to achieve a trade-off, thereby avoiding a full codebook attack. Consequently, the overall complexity of our attack is $\mathcal{D} = 2^{47}$, $\mathcal{T} = 2^{95.31}$ and $\mathcal{M} = 2^{96.14}$.

5 Application on Piccolo-128

In this section, we present our results on Piccolo-128. We first develop an automatic tool for searching full differential MITM attacks for Piccolo. We then present the obtained attacks on 15 and 14 rounds. The description of Piccolo is provided in Appendix E.

5.1 Automatic Tool

We developed a two-stage MILP-based automatic search tool to identify efficient differential trails for differential MITM attacks against Piccolo. Although our tool is modeled specifically for Piccolo, the search framework can be applied to other word-oriented ciphers. In the first stage, we construct a word-wise model where each bit represents whether a word is active. This stage allows us to determine the number of active S-boxes and the number of key bytes involved in the

key recovery phase. Since the maximum probability of a **Piccolo** S-box is 2^{-2} , our objective function to minimize in this stage is $2 \times \# \text{S-box} + \max(|k_{in}|, |k_{out}|)$.

In the second stage, we use the solution from the first stage as constraints, and construct a bitwise model to search for the differential trail with the maximum probability. Additionally, we introduce specialized constraints based on Matsui’s bounding condition, as proposed in [25], to accelerate the search process. The source code for our model is available at <https://github.com/d-mitm/Piccolo>.

5.2 Differential MITM Attacks on 15-Round **Piccolo**-128

This attack uses a distinguisher consisting of only 4 rounds, instead of a long differential trail. This allows us to guess more key bits during the key guessing phase, resulting in a more efficient attack. Table 3 shows the differential trail used in our attack. We verified the 4 differential transitions in this trail by evaluating all 2^{16} inputs of the F -function. Assuming the Markov cipher model, we estimate the probability of this trail as $2^{-(9.83+11+9.83+11)} = 2^{-41.66}$.

Table 3. The 4-round differential trail of **Piccolo** in the single-key setting.

#Round	State difference	#Active S-boxes	Diff. transition	$-\log_2(p)$
0	8880 0000 0000 D100	5	$0x8880 \xrightarrow{F} 0x0042$	9.83
1	0000 0080 D142 8800	5	$0xD142 \xrightarrow{F} 0x0100$	11
2	0000 D100 8980 0042	5	$0x8980 \xrightarrow{F} 0x0052$	9.83
3	D110 8900 0000 0080	5	$0xD110 \xrightarrow{F} 0x8900$	11
4	D110 0000 0000 0080	-	-	-

To exploit the key schedule of **Piccolo**-128, the attack targets rounds 11 to 25, covering a total of 15 rounds. We represent the internal states as S_i where $i = 10, 11, \dots, 25$. Note that S_{10} corresponds to the plaintext P , and S_{25} corresponds to the ciphertext C in this attack. Each state S_i consists of 8 bytes, denoted as $S_i[0, 1, \dots, 7]$. There are 5 rounds on top of the 4-round differential trail, followed by 4 additional rounds at the end. By employing the parallel partitioning technique, we extend the attack by two additional rounds. Fig. 10 illustrates our attack on **Piccolo**-128 without parallel partitions. Instead of XORing the state bytes with the round key, we bypass the constant and directly XOR them with part of the master key⁵ $K_i = K[16i, 16i + 15]$ for $i = 0, 1, \dots, 7$. This approach leverages the straightforward key schedule of **Piccolo**-128. We denote the left and right bytes of K_i as K_i^L and K_i^R , respectively. Our MILP program automatically identifies all the key bytes involved in the key recovery phase. These are as follows: $k_{in} = \{K_0, K_1, K_3, K_4, K_5^R, K_6^L\}$, $k_{out} = \{K_0, K_3, K_4, K_5^L, K_6^R, K_7\}$.

⁵ $K[0, 1, \dots, 127]$ denotes the master key.

Attack Procedure.

1. Ask for the encryption of the whole codebook.
2. Fix $S_{23}[2, 3, 6, 7]$ as a constant, as depicted in Fig. 11. Enumerate all possible $S_{23}[0, 1, 4, 5]$ to construct a set of size 2^{32} .
3. Guess 48-bit $k_\cap = k_{in} \cap k_{out} = \{K_0, K_3, K_4\}$. Do the following steps:
 - For each S_{23} in the current set and each of the $2^{|k_{out}| - |k_\cap|} = 2^{80-48} = 2^{32}$ possible values j of $k_{out} \setminus k_\cap = \{K_7, K_5^L, K_6^R\}$, generate $2^{32+32} = 2^{64}$ tuples in the form of $(S_{23}, \tilde{S}_{23}, j)$ such that they can meet Δ_{out} at the end of the distinguisher. Store all these tuples in a hash table.
 - Since **Piccolo** is a cipher with an underlying structure of GFN (Generalized Feistel Network), i.e., half of its state bits remain the same after one round, we can construct a set of S_{24} that corresponds to the set of S_{23} . Specifically, we set $S_{24}[0] = S_{23}[2], S_{24}[1] = S_{23}[7], S_{24}[4] = S_{23}[6]$, and $S_{24}[5] = S_{23}[3]$, then enumerate all possible $S_{24}[2, 3, 6, 7]$ to form a set. With the knowledge of $\{K_3, K_4\} \subset k_\cap$, we can encrypt all the S_{24} to ciphertext S_{25} , i.e., C . Then, obtain all the plaintexts from the codebook. For each P (i.e., S_{10}) in the set and each of the $2^{|k_{in}| - |k_\cap|} = 2^{80-48} = 2^{32}$ possible values i of $k_{in} \setminus k_\cap = \{K_1, K_5^R, K_6^L\}$, generate $2^{32+32} = 2^{64}$ tuples in the form of $(P, \tilde{P}, i)$ such that they can meet Δ_{in} at the beginning of the distinguisher.
 - From the codebook, we obtain 2^{64} tuples $(C = E(P), \tilde{C} = E(\tilde{P}), i)$. We then decrypt them to $(S_{24}, \tilde{S}_{24}, i)$ with the knowledge of $\{K_3, K_4\}$. Applying the parallel partitioning technique, we match $(S_{24}, \tilde{S}_{24}, i)$ and $(S_{23}, \tilde{S}_{23}, j)$ through the equations depicted in Table 4. In this way, we will obtain $2^{64+64-96} = 2^{32}$ key candidates. We then guess 2^{16} possible $\{K_2\} = K \setminus (k_{in} \cup k_{out})$, and test each candidate with an extra plaintext-ciphertext pair to determine whether it is the right key.
4. Go to step 2 and fix $S_{23}[2, 3, 6, 7]$ with a different constant until the right key is found. The above procedure is expected to repeat for $2^{p-m} = 2^{41.66-32} = 2^{9.66}$ times.

Table 4. Relations between $(S_{24}, \tilde{S}_{24})$ and $(S_{23}, \tilde{S}_{23})$.

#Index	Equations	Filter bits
1	$S_{23}[2] = S_{24}[0] \oplus K_6^L, \tilde{S}_{23}[2] = \tilde{S}_{24}[0] \oplus K_6^L$	16
2	$S_{23}[3] \oplus K_6^R = S_{24}[5], \tilde{S}_{23}[3] \oplus K_6^R = \tilde{S}_{24}[5]$	16
3	$S_{23}[6] = S_{24}[4] \oplus K_1^L, \tilde{S}_{23}[6] = \tilde{S}_{24}[4] \oplus K_1^L$	16
4	$S_{23}[7] = S_{24}[1] \oplus K_1^R, \tilde{S}_{23}[7] = \tilde{S}_{24}[1] \oplus K_1^R$	16
5	$\tilde{S}_{23}[0] = \tilde{S}_{24}[6]$	8
6	$\tilde{S}_{23}[1] = \tilde{S}_{24}[3]$	8
7	$\tilde{S}_{23}[4] = \tilde{S}_{24}[2]$	8
8	$\tilde{S}_{23}[5] = \tilde{S}_{24}[7]$	8

One bit fewer in k_{out} . We observe that it is possible to guess one bit fewer in k_{out} . Leveraging our differential trail, the input difference of the active F -function in round 20 is 0x0080, meaning that only one bit is active. Therefore, the value of $K_6^R[0]$ does not influence the output difference of this F -function. Consequently, we can still obtain $\tilde{S}_{23}$ without $K_6^R[0]$, which means that $|k_{out}|$ is actually 79.

Complexity. The time complexity is $\mathcal{T} = 2^{9.66} \times 2^{48} \times (2^{32+32} \times \frac{2 \times 5}{15} + 2^{32+31} \times \frac{2 \times 4}{15} + 2^{31+17}) = 2^{121.56}$. Since the whole codebook is needed, the data complexity is $\mathcal{D} = 2^{64}$. The memory complexity is $\mathcal{M} = 2^{64} \times 16 = 2^{68}$ bytes.

Data-time-memory trade-off. As explained in 2.3, we can force 4 bits in the plaintext⁶ to achieve a trade-off. Consequently, the overall complexities of our attack are: $\mathcal{D} = 2^{60}$, $\mathcal{T} = 2^{125.56}$, and $\mathcal{M} = 2^{64}$.

5.3 A Lower-data Attack against 14-round Piccolo-128

As explained in Sect. 3.1, the flexible key recovery strategy can be applied to differential MITM attacks when combined with the parallel partitioning technique. By extending the same differential trail, as listed in Table 3, with 5 rounds before and 4 rounds after, we present an attack on 14-round Piccolo-128 with lower data complexity.

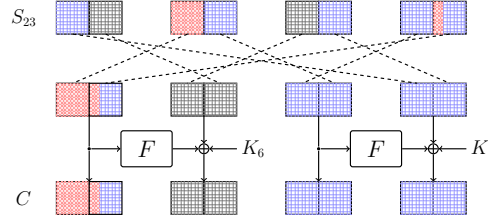


Fig. 6. The last round of the 14-round improved attack on Piccolo-128. Red bits are fixed in advance; blue bits represent c'_{ex} ; gray bits represent c_{ex}^* .

Attack Procedure.

1. As shown in Fig. 6, fix the leftmost $\lfloor (n-p-1)/2 \rfloor = 10$ bits in the ciphertext C . Ask for plaintexts for all $2^s = 2^{n-10} = 2^{54}$ ciphertexts, and collect all tuples (P, C) to construct a structure ST_1 . Fix $S_{23}[2]$ and the leftmost 2 bits in $S_{23}[7]$ with the same constant, and construct a structure ST_2 of S_{23} .
2. Guess $k'_\cap = \{K_0, K_3, K_4\}$.

⁶ $4 < \frac{n-p-1}{2}$

- Guess all possible i for $k'_{in} \setminus k'_\cap = \{K_1\}$ and partially encrypt all plaintexts in ST_1 to S_{13} . Combining with the differential pattern in S_{14} , we can obtain $c'_{in} = 48$ filtering bits from the following 4 equations:
 - (1) $S_{13}[0] \oplus \tilde{S}_{13}[0] = 0x00$;
 - (2) $S_{13}[2, 3] \oplus \tilde{S}_{13}[2, 3] \oplus F(S_{13}[0, 1]) \oplus F(\tilde{S}_{13}[0, 1]) = 0xD100$;
 - (3) $S_{13}[5] \oplus \tilde{S}_{13}[5] = 0x80$;
 - (4) $S_{13}[6, 7] \oplus \tilde{S}_{13}[6, 7] \oplus F(S_{13}[6, 7]) \oplus F(\tilde{S}_{13}[6, 7]) = 0x0000$.
 Thus, we will get $\binom{2^s}{2} \times 2^{|k'_{in} \setminus k'_\cap| - c'_{in}} = 2^{75}$ pairs $(P, \tilde{P})$. Store the corresponding $(C, \tilde{C}, i)$ in a hash table.
- Guess all possible j for $k'_{out} \setminus k'_\cap = \{K_7\}$ and partially decrypt all S_{23} in ST_2 to S_{21} . Combining with the differential pattern in S_{20} , we can obtain $c'_{out} = 48$ filtering bits from the following 4 equations:
 - (1) $S_{21}[1] \oplus \tilde{S}_{21}[1] = 0x00$;
 - (2) $S_{21}[2, 3] \oplus \tilde{S}_{21}[2, 3] \oplus F(S_{21}[0, 1]) \oplus F(\tilde{S}_{21}[0, 1]) = 0x0080$;
 - (3) $S_{21}[4] \oplus \tilde{S}_{21}[4] = 0xD1$;
 - (4) $S_{21}[6, 7] \oplus \tilde{S}_{21}[6, 7] \oplus F(S_{21}[4, 5]) \oplus F(\tilde{S}_{21}[4, 5]) = 0x0000$.
 Thus, we will get $\binom{2^s}{2} \times 2^{|k'_{out} \setminus k'_\cap| - c'_{out}} = 2^{75}$ pairs $(S_{23}, \tilde{S}_{23}, j)$.
- Check for possible matches in the hash table. Since we have the knowledge of K_1 that belongs to k'_{in} , the match is performed on the equations below:

$$\begin{aligned}
 C[1] &= S_{23}[7], \tilde{C}[1] = \tilde{S}_{23}[7], C[4] = S_{23}[6], \tilde{C}[4] = \tilde{S}_{23}[6], \\
 C[5] &= S_{23}[3], \tilde{C}[5] = \tilde{S}_{23}[3], C[6] \oplus K_1^L = S_{23}[0], \tilde{C}[6] \oplus K_1^L = \tilde{S}_{23}[0], \\
 C[7] \oplus K_1^R &= S_{23}[5], \tilde{C}[7] \oplus K_1^R = \tilde{S}_{23}[5], \\
 C[2] \oplus \tilde{C}[2] &= S_{23}[4] \oplus \tilde{S}_{23}[4], C[3] \oplus \tilde{C}[3] = S_{23}[1] \oplus \tilde{S}_{23}[1].
 \end{aligned}$$
 Since the left two bits in $C[1]$ and $S_{23}[7]$ are already fixed as the same value, the quantities above represent a 92-bit filter. The number of expected collisions is $2^{75+75-92} = 2^{58}$.
- For each collision, extract the other bytes of k_{in} and k_{out} :
 - First, we compute ΔS_{14} from S_{13} and $\tilde{S}_{13}$. From $0xD100 \xrightarrow{F} \Delta S_{14}[2] \oplus 0x88 \parallel \Delta S_{14}[3]$, we can obtain $k_{in}^* = \{K_5^R, K_6^L\}$.
 - Similarly, we compute ΔS_{20} from S_{21} and $\tilde{S}_{21}$. From $0x0080 \xrightarrow{F} \Delta S_{20}[2] \parallel \Delta S_{20}[3] \oplus 0x10$, we can obtain $k_{out}^* = \{K_5^L, K_6^R\}$.
 - Check K_6 by $C[2] \oplus S_{23}[4] = K_6^L$ and $C[3] \oplus S_{23}[1] = K_6^R$.
 We are expected to extract $2^{|k_{in}^* \cup k_{out}^*| - c_{in}^* - c_{out}^* - c_{ex}^*} = 2^{16+15-16-16-16} = 2^{-17}$ key candidates. For each key candidate, exhaust K_2 and perform an extra test.

Correctness and Complexity. Since we prepare a structure of size 2^{54} , for the correct key guess, we expect to find $\binom{2^s}{2} \times 2^{-n-p} = 2^{54+54-1-64-41.66} = 2^{1.34} > 2$ right pairs. Finally, we can recover the master key by exhaustively searching the remaining key bits.

The time complexity is $\mathcal{T} = 2^{54} + 2^{48} \times (2^{75} + 2^{75} + 2^{58-17+16}) = 2^{124}$. The data complexity is $\mathcal{D} = 2^{54}$. The memory complexity is $\mathcal{M} = 2^{75} \times 18 = 2^{79.17}$ bytes for storing the hash table.

6 Conclusion

We improved the differential MITM attack by proposing a generalized framework that combines the flexible key guessing strategy with the improved parallel partitioning technique. We have applied our methods to the attacks on **SIMON** and **Piccolo**, and obtained favorable results. In particular, we achieved a 26-round key recovery attack on **SIMON48/96**, which represents the best attack in the single-key scenario. We also developed an MILP-based tool for searching full differential MITM attacks on **Piccolo**.

We see several interesting avenues opening up from here. We hope to develop a generalized automatic tool that incorporates all the current techniques, aiming for SPN and Feistel ciphers. The differential MITM attacks in the related-key setting may provide valuable insights into the security margins of block ciphers.

Acknowledgments

The authors would like to thank the anonymous reviewers for their valuable suggestions and comments. This work is supported by National Natural Science Foundation of China (Grant No. 62302293), and National Key R&D Program of China (Grant No. 2024YFA1013000).

References

1. Abdelraheem, M.A., Alizadeh, J., AlKhazaimi, H.A., Aref, M.R., Bagheri, N., Gauravaram, P.: Improved linear cryptanalysis of reduced-round **SIMON**-32 and **SIMON**-48. In: Biryukov, A., Goyal, V. (eds.) **INDOCRYPT 2015**. LNCS, vol. 9462, pp. 153–179. Springer, Cham (Dec 2015). https://doi.org/10.1007/978-3-319-26617-6_9
2. Advanced Encryption Standard (AES). National Institute of Standards and Technology, NIST FIPS PUB 197, U.S. Department of Commerce (Nov 2001)
3. Ahmadian, Z., Khalesi, A., M’foukh, D., Moghimi, H., Naya-Plasencia, M.: Improved differential meet-in-the-middle cryptanalysis. In: Joye, M., Leander, G. (eds.) **EUROCRYPT 2024**, Part I. LNCS, vol. 14651, pp. 280–309. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58716-0_10
4. Azimi, S.A., Ahmadian, Z., Mohajeri, J., Aref, M.R.: Impossible differential cryptanalysis of piccolo lightweight block cipher. In: 11th International ISC Conference on Information Security and Cryptology, **ISCISC 2014**. pp. 89–94. IEEE (2014), <https://doi.org/10.1109/ISCISC.2014.6994028>
5. Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., Wingers, L.: The **SIMON** and **SPECK** families of lightweight block ciphers. *Cryptology ePrint Archive*, Report 2013/404 (2013), <https://eprint.iacr.org/2013/404>
6. Beierle, C., Jean, J., Kölbl, S., Leander, G., Moradi, A., Peyrin, T., Sasaki, Y., Sasdrich, P., Sim, S.M.: The **SKINNY** family of block ciphers and its low-latency variant **MANTIS**. In: Robshaw, M., Katz, J. (eds.) **CRYPTO 2016**, Part II. LNCS, vol. 9815, pp. 123–153. Springer, Berlin, Heidelberg (Aug 2016). https://doi.org/10.1007/978-3-662-53008-5_5

7. Biham, E., Shamir, A.: Differential cryptanalysis of DES-like cryptosystems. In: Menezes, A.J., Vanstone, S.A. (eds.) CRYPTO'90. LNCS, vol. 537, pp. 2–21. Springer, Berlin, Heidelberg (Aug 1991). https://doi.org/10.1007/3-540-38424-3_1
8. Boura, C., David, N., Derbez, P., Leander, G., Naya-Plasencia, M.: Differential meet-in-the-middle cryptanalysis. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part III. LNCS, vol. 14083, pp. 240–272. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38548-3_9
9. Boura, C., Derbez, P., Funk, M.: Related-key differential analysis of the AES. *IACR Trans. Symm. Cryptol.* **2023**(4), 215–243 (2023). <https://doi.org/10.46586/tosc.v2023.i4.215-243>
10. Chen, H., Wang, X.: Improved linear hull attack on round-reduced simon with dynamic key-guessing techniques. In: Peyrin, T. (ed.) FSE 2016. LNCS, vol. 9783, pp. 428–449. Springer, Berlin, Heidelberg (Mar 2016). https://doi.org/10.1007/978-3-662-52993-5_22
11. Grochal, P., Stanek, M.: Differential MITM attacks on SLIM and LBCIoT. *Cryptology ePrint Archive*, Paper 2024/1887 (2024), <https://eprint.iacr.org/2024/1887>
12. Isobe, T., Shibutani, K.: Security analysis of the lightweight block ciphers XTEA, LED and Piccolo. In: Susilo, W., Mu, Y., Seberry, J. (eds.) ACISP 12. LNCS, vol. 7372, pp. 71–86. Springer, Berlin, Heidelberg (Jul 2012). https://doi.org/10.1007/978-3-642-31448-3_6
13. Kölbl, S., Leander, G., Tiessen, T.: Observations on the SIMON block cipher family. In: Gennaro, R., Robshaw, M.J.B. (eds.) CRYPTO 2015, Part I. LNCS, vol. 9215, pp. 161–185. Springer, Berlin, Heidelberg (Aug 2015). https://doi.org/10.1007/978-3-662-47989-6_8
14. Lin, H., Zou, J., Li, J.: The differential meet-in-the-middle attack on FUTURE and CRAFT. In: *Proceedings of the 2023 13th International Conference on Communication and Network Security, ICCNS 2023*. pp. 151–158. ACM (2023), <https://doi.org/10.1145/3638782.3638805>
15. Liu, Y., Cheng, L., Liu, Z., Li, W., Wang, Q., Gu, D.: Improved meet-in-the-middle attacks on reduced-round piccolo. *Sci. China Inf. Sci.* **61**(3), 032108:1–032108:13 (2018), <https://doi.org/10.1007/s11432-016-9157-y>
16. Liu, Y., Cheng, L., Zhao, F., Su, C., Liu, Z., Li, W., Gu, D.: New analysis of reduced-version of piccolo in the single-key scenario. *KSII Trans. Internet Inf. Syst.* **13**(9), 4727–4741 (2019), <https://doi.org/10.3837/tiis.2019.09.022>
17. Liu, Z., Li, Y., Wang, M.: Optimal differential trails in SIMON-like ciphers. *IACR Trans. Symm. Cryptol.* **2017**(1), 358–379 (2017). <https://doi.org/10.13154/tosc.v2017.i1.358-379>
18. Qiao, K., Hu, L., Sun, S.: Differential analysis on simeck and SIMON with dynamic key-guessing techniques. In: *Information Systems Security and Privacy - Second International Conference, ICISSP 2016. Communications in Computer and Information Science*, vol. 691, pp. 64–85. Springer (2016), https://doi.org/10.1007/978-3-319-54433-5_5
19. Shibutani, K., Isobe, T., Hiwatari, H., Mitsuda, A., Akishita, T., Shirai, T.: Piccolo: An ultra-lightweight blockcipher. In: Preneel, B., Takagi, T. (eds.) CHES 2011. LNCS, vol. 6917, pp. 342–357. Springer, Berlin, Heidelberg (Sep / Oct 2011). https://doi.org/10.1007/978-3-642-23951-9_23
20. Song, L., Liu, H., Yang, Q., Chen, Y., Hu, L., Weng, J.: Generic differential key recovery attacks and beyond. In: Chung, K.M., Sasaki, Y. (eds.) ASI-

- ACRYPT 2024, Part VII. LNCS, vol. 15490, pp. 361–391. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0941-3_12
21. Song, L., Yang, Q., Liu, H.: Revisiting the differential meet-in-the-middle cryptanalysis. Cryptology ePrint Archive, Report 2023/1302 (2023), <https://eprint.iacr.org/2023/1302>
 22. Tolba, M., Abdelkhalek, A., Youssef, A.M.: Meet-in-the-middle attacks on reduced round piccolo. In: Lightweight Cryptography for Security and Privacy - 4th International Workshop, LightSec 2015. Lecture Notes in Computer Science, vol. 9542, pp. 3–20. Springer (2015), https://doi.org/10.1007/978-3-319-29078-2_1
 23. Wang, N., Wang, X., Jia, K., Zhao, J.: Differential attacks on reduced SIMON versions with dynamic key-guessing techniques. Sci. China Inf. Sci. **61**(9), 098103:1–098103:3 (2018), <https://doi.org/10.1007/s11432-017-9231-5>
 24. Zhang, Y., Lyu, L., Qiao, K., Zhang, Z., Sun, S., Hu, L.: Automatic key recovery of feistel ciphers: Application to SIMON and SIMECK. In: Information Security Practice and Experience: 16th International Conference, ISPEC 2021. Lecture Notes in Computer Science, vol. 13107, pp. 147–167. Springer (2021), https://doi.org/10.1007/978-3-030-93206-0_10
 25. Zhang, Y., Sun, S., Cai, J., Hu, L.: Speeding up MILP aided differential characteristic search with matsui’s strategy. In: Information Security - 21st International Conference, ISC 2018, Guildford, UK, September 9-12, 2018, Proceedings. Lecture Notes in Computer Science, vol. 11060, pp. 101–115. Springer (2018), https://doi.org/10.1007/978-3-319-99136-8_6

A Algorithm for the Improved Differential MITM Attack

Algorithm 1 The improved differential MITM attack.

Require: $|\Delta_P| = |\Delta_S| = n$

- 1: Fix $x = \frac{n-p-1}{2}$ bits in ciphertext C
 - 2: Construct two structures: $ST_1 \leftarrow \{(P, C)\}$ and $ST_2 \leftarrow \{S\}$
 - 3: **for** each possible v for $k'_\cap$ **do**
 - 4: **for** each possible i for $k'_{in} \setminus k'_\cap$ **do** ▷ Upper part
 - 5: Do partial encryption for each P in ST_1
 - 6: Obtain $\binom{2^{n-x}}{2} \times 2^{-c'_{in}}$ pairs $(P, \tilde{P})$
 - 7: Store the corresponding $(C, \tilde{C}, i)$ in a hash table H
 - 8: **end for**
 - 9: **for** each possible j for $k'_{out} \setminus k'_\cap$ **do** ▷ Lower part
 - 10: Do partial decryption for each S in ST_2
 - 11: Obtain $\binom{2^{n-x'}}{2} \times 2^{-c'_{out}}$ pairs $(S, \tilde{S})$ and find collisions in H
 - 12: **for** each collision **do** ▷ After a c'_{ex} -bit filter
 - 13: Get (v, i, j) and extract $2^{|k_{in}^* \cup k_{out}^*| - c_{in}^* - c_{out}^* - c_{ex}^*}$ candidates for $k_{in}^* \cup k_{out}^*$
 - 14: Update the key counters or guess the remaining key bits and test directly
 - 15: **end for**
 - 16: **end for**
 - 17: **end for**
-

B Figure for Typical Two-round Feistel Structure of E_{ex}

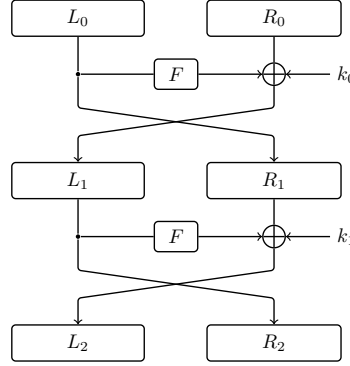


Fig. 7. Typical two-round Feistel structure of E_{ex}

C Description of SIMON

SIMON is a family of lightweight block ciphers designed by the U.S. National Security Agency (NSA). It was introduced in 2013 as part of a design competition for lightweight cryptography aimed at devices with limited computational resources, such as IoT devices and embedded systems.

SIMON is a substitution-permutation network (SPN) cipher that uses bit operations such as XOR, AND, and circular shifts. Table 5 lists the block size, the key size and the number of rounds for various versions of SIMON.

Table 5. Parameters for SIMON-family

block size n	key size K	rounds T
32	64	32
48	72	36
	96	36
64	96	42
	128	44
96	96	52
	144	54
128	128	68
	192	69
	256	72

The r -th round function of **SIMON** is a Feistel map: $(X_L^{r-1}, X_R^{r-1}) \rightarrow (X_L^r, X_R^r)$ where $X_R^r = X_L^{r-1}$ and $X_L^r = ((X_L^{r-1} \lll 8) \& (X_L^{r-1} \lll 1)) \oplus (X_L^{r-1} \lll 2) \oplus X_R^{r-1} \oplus k^{r-1}$. The round function of **SIMON** is depicted in Fig. 8.

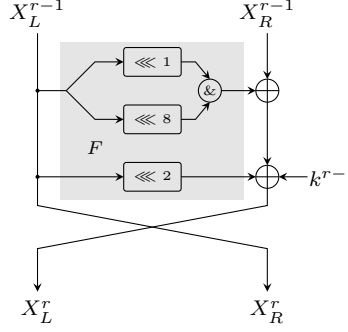


Fig. 8. Round Function of **SIMON**

For **SIMON** $n/2n$ with key size $K = 2n$, let $c = 2^{n/2} - 4 = 0\mathbf{x}\mathbf{ff} \cdots \mathbf{fc}$ and z_j be a constant sequence. The round keys are generated by

$$k^{i+4} = c \oplus (z_j)_i \oplus k^i \oplus (k^{i+3} \ggg 3) \oplus (k^{i+3} \ggg 4) \oplus k^{i+1} \oplus (k^{i+1} \ggg 1)$$

for $0 \leq i < T - 4$. The round keys used in round 0 to 3 are derived from the master key. We refer to [5] for more details.

D More Details of Attacks on **SIMON** introduced in Section 4

D.1 Subkey Bits Involved in Attacks against **SIMON**

Table 6. Subkey bits involved in the 23-round attack against **SIMON**32/64 proposed in Sect. 4.1.

Part	Involved key bits
k_{in}	$k_0[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]$
	$k_1[1, 5, 6, 7, 8, 9, 10, 11, 15]$
	$k_2[7, 9]$
k_{out}	$k_{19}[1, 3]$
	$k_{20}[0, 1, 2, 3, 4, 5, 9, 11, 15]$
	$k_{21}[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]$

Table 7. Subkey bits involved in the 26-round attack against SIMON48/96 proposed in Sect. 4.3.

Part	Involved key bits
k_{in}	$k_0[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23]$
	$k_1[0, 1, 5, 7, 11, 12, 14, 15, 17, 21, 22]$
	$k_2[13, 23]$
k_{out}	$k_{21}[1, 7, 11, 15, 17, 21]$
	$k_{22}[0, 1, 2, 3, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 16, 17, 18, 19, 20, 22, 23]$
	$k_{23}[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23]$

D.2 Differential MITM attack against 32-round SIMON64/128

Distinguisher. As we do in Sect. 4.3, we employ the CryptoSMT tool to find a 22-round differential $(0x4, 0x11) \rightarrow (0x40, 0x11)$ for SIMON64 and calculate its probability. We sum up over 200,000 trails with probability from 2^{-72} to 2^{-81} and obtain $2^{-p} = 2^{-61.72}$ as shown in Table 8.

Table 8. Overview of the differential and the range $[w_{min}, w_{max}]$ of the $-\log_2$ probabilities of the characteristics contributing to the differential.

Rounds	Δ_{in}	Δ_{out}	w_{min}	w_{max}	$\log_2(p)$
22	$(0x4, 0x11)$	$(0x40, 0x11)$	72	81	-61.72

Key Guessing Part. We add 4 rounds before the distinguisher and 4 rounds after. k_{in} and k_{out} in this attack are listed in Table 9.

Table 9. Subkey bits involved in the 32-round attack.

Part	k_r	Indices of involved key bits in k_r
k_{in}	k_0	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31
	k_1	1, 3, 5, 6, 8, 9, 10, 12, 13, 15, 17, 19, 21, 23, 24, 26, 27, 28, 30, 31
	k_2	7, 11, 25, 29
	k_3	$\emptyset$
k_{out}	k_{26}	$\emptyset$
	k_{27}	13, 31
	k_{28}	0, 1, 5, 7, 11, 12, 14, 15, 21, 23, 25, 29, 30
	k_{29}	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31

We use the same method as in Sect. 4.1 to verify that $|k_{in}| = 56$, $|k_{out}| = 46$, $|k_{re}| = 26$, and that $k_{in} \cap k_{out} = \emptyset$. Thus we can set $|\alpha| = 7$ and $|\beta| = 19$. By applying a similar attack as described in Sect. 4.2, we can append two additional rounds, resulting in a 32-round attack.

Complexity.

- The data complexity $\mathcal{D} = 2^{64}$;
- The time complexity $\mathcal{T} = 2^{64+56+7} \times \frac{8}{32} + 2^{61.72+46+19} \times \frac{8}{32} + 2^{125.72} = 2^{126.8}$ encryptions;
- The memory complexity $\mathcal{M} = 2^{127}$ 191-bit words.

Data-time-memory trade-off. As explained in 2.3, we can query the oracle for a subset of the entire codebook to avoid a full codebook attack. After the trade-off, the overall complexities of our attack are: $\mathcal{D} = 2^{63.8}$, $\mathcal{T} = 2^{127}$ and $\mathcal{M} = 2^{131.4}$.

E Description of Piccolo

Piccolo is a 64-bit block cipher supporting 80 and 128-bit keys. Two key modes are referred to as Piccolo-80 and Piccolo-128, respectively. Both ciphers consist of an encryption algorithm and a key schedule algorithm.

Encryption Algorithm. The general structure of Piccolo is a variant of the Generalized Feistel Network, which is depicted in Fig. 9. The number of iterative rounds is 25 for Piccolo-80 and 31 for Piccolo-128. Each round is made up of two functions $F : \{0, 1\}^{16} \rightarrow \{0, 1\}^{16}$ and one round permutation $RP : \{0, 1\}^{64} \rightarrow \{0, 1\}^{64}$. F consists of two S-box layers separated by a diffusion matrix M .

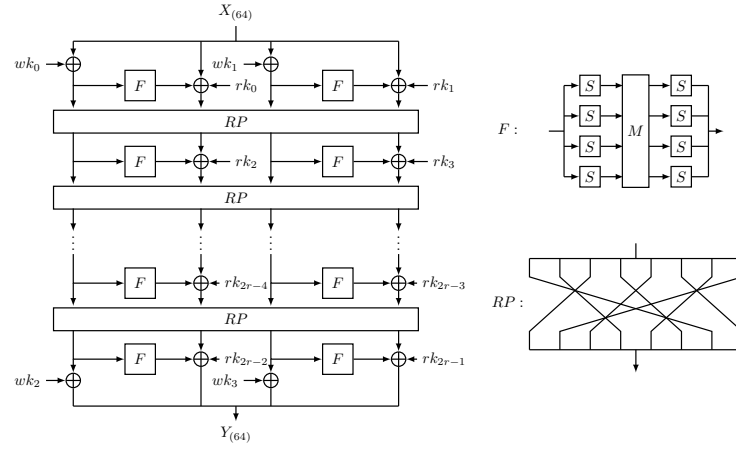
$$M = \begin{pmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{pmatrix}$$

RP divides a 64-bit input into eight bytes and then permutes them.

Key Schedule Algorithm. For Piccolo-128, rk_{2r-2} and rk_{2r-1} are round keys introduced into the r -th round. Each 16-bit round key rk_i is derived from a 128-bit secret key $K = K_0 || K_1 || \dots || K_7$ according to the predefined manner (see Table 10, e.g., $rk_0 = K_2, rk_1 = K_3$).

We refer to [19] for more details.

F Details for Differential MITM Attacks on 15-Round Piccolo-128 in Sect. 5.2

**Fig. 9.** Description of Piccolo**Table 10.** Key expanding function of Piccolo-128

Round r (1 to 16)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$(rk_{2r-2} rk_{2r-1})$	2	3	4	5	6	7	2	1	6	7	0	3	4	5	6	1
Round r (17 to 31)	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	
$(rk_{2r-2} rk_{2r-1})$	2	7	4	3	6	5	2	1	6	5	0	7	4	3	6	1

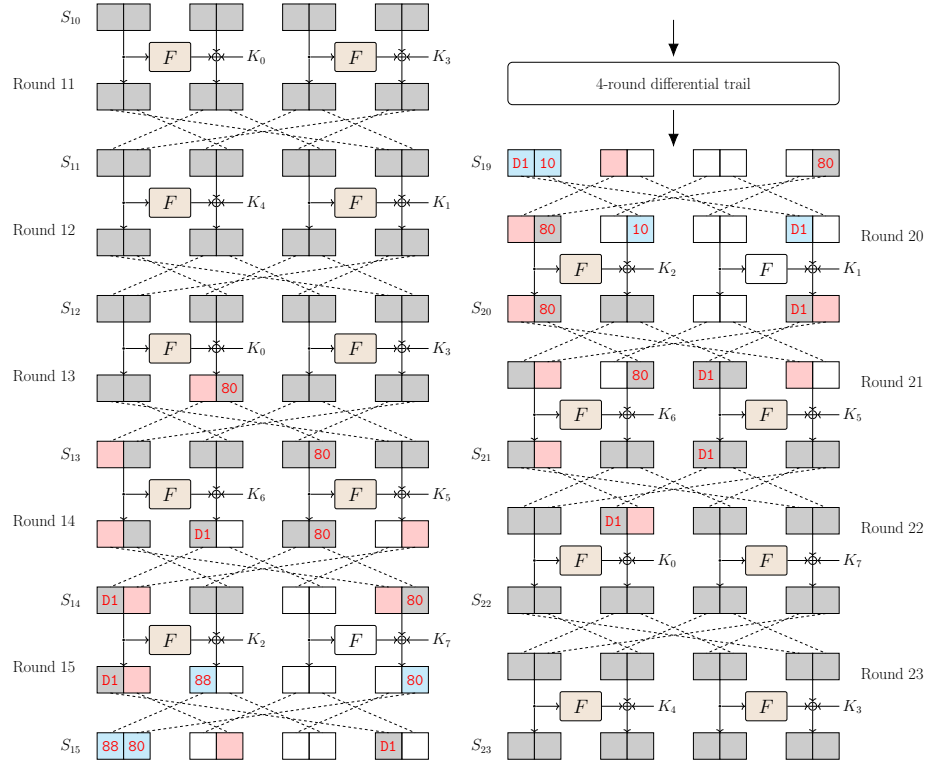


Fig. 10. Differential MITM attack against 15-round Piccolo-128. Colored F -functions are active, whereas white F -functions are inactive. Gray bytes (active) and red bytes (inactive) both have known values; blue bytes (active) and white bytes (inactive) both have unknown values.

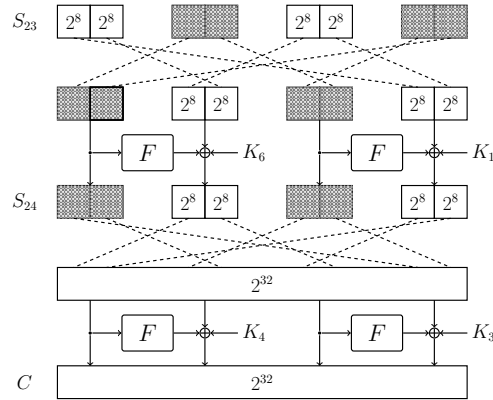


Fig. 11. The last two rounds of the 15-round attack on Piccolo-128. Hashed bytes are fixed in advance.